

Lexer (Flex) Practice Problems

BUET CSE 310 – Compiler Sessional Prep

Focus: Extending 2205014.1 Lexer Implementation

General Exam Strategy & Guidelines

Exam practice focus: Since exams almost always build upon your existing lexer, practicing finding rules, adding patterns, and protecting longest-match behavior is the highest-value prep. Keep these rules in mind:

- **Read the complete prompt:** Pay attention to specific output formats mentioned in sub-paragraphs.
- **Avoid longest-match collisions:** Plan regex interactions before coding (e.g., += vs ADDOP / ASSIGNOP).
- **Choose correct state management:** Use exclusive start conditions for multi-line or delimited structures (like comments and strings).
- **Handle EOF:** Always define <<EOF>> behavior for custom flex states.

Warm-ups (10–15 min)

1. Max scope depth

Track the deepest nesting level reached during the whole file (i.e., the maximum number of { you are inside at once) and print:

Maximum scope depth: N

at the end of the log file, right after Total errors.

2. New keywords

Add struct, const, signed, and sizeof as keywords, producing <STRUCT>, <CONST>, <SIGNED>, and <SIZEOF> respectively, following the rule that keywords are not inserted into the symbol table.

Core Problems (30–45 min each)

3. Compound assignment operators

Add +=, -=, *=, /=, %= as token type ASSIGNOP (same token type as =, different lexeme).

Example:

```
1 arr[0] += 5;
```

Expected Output: Line no 1: Token <ID> Lexeme arr found ... Line no 1: Token <ASSIGNOP> Lexeme += found ...

Ensure that x +=5 (with whitespace) tokenizes separately as ADDOP then ASSIGNOP, while x+=5 scans cleanly as ASSIGNOP.

4. Bitwise operators

Add &, |, ^, ~, <<, >> as a new token type BITOP.

Watch out: Single & vs && (LOGICOP), and <</>> vs </> (RELOP). Test with:

```
1 int mask = a & b | (c << 2);
```

5. Ternary operator

Add ? → QUESTION and : → COLON as new punctuator tokens. Test with:

```
1 int max = (a > b) ? a : b;
```

6. Hexadecimal & octal integers

Recognize 0x1A3F (hex) and 0755 (octal, leading 0 followed only by octal digits 0--7) as CONST_INT. Insert them into the symbol table using the original lexeme string.

Add error reporting:

- Octal literal containing 8 or 9 (e.g., 0783) → report `Invalid octal digit`.
- Hex literal with non-hex character (e.g., 0x1G) → report `Invalid hex digit`.

Note: Consider interaction with existing INVALID_ID_SUFFIX rules.

7. Identifier length limit

Identifiers longer than 20 characters are invalid. Recognize them lexically as identifiers (without breaking tokenization), but log:

```
Line no X: Identifier length exceeds limit: <name>
```

as an error instead of standard symbol table insertion/printing.

8. Invalid escape sequence in strings

Add detection for unknown escapes (e.g., \q). If encountered, report:

```
Invalid escape sequence '\q' in string
```

at that line, but continue scanning the string normally without marking it unterminated. Test with:

```
1 char *s = "hello\qworld";
```

9. Preprocessor directive

Add a new exclusive start condition to capture lines beginning with # (e.g., #include <stdio.h>, #define MAX 100) as a single PREPROCESSOR entry. Log it like comments (no token file entry), supporting line continuation using \. Ensure this takes priority over unrecognized character rules.

10. Nested comment warning

If /* appears inside a multi-line comment, maintain normal comment scanning (no nested comments) but log a warning:

```
Line no X: Warning - nested comment start ignored
```

Continue scanning until the closing */.

11. Array size in the symbol table

When detecting ID [CONST_INT] in declaration context (e.g., int arr[50];), attach array size to the symbol table entry: < arr : ID, size=50 >. Standard accesses like arr[0] = 5; should NOT re-tag the symbol.

12. Line numbers in the token file

Modify the token file to include line numbers in every token format: <INT, 2>, <ID, x, 2>, <CONST_INT, 672, 2 >, including zero – *lexemekeywordtokens*.